NeuroAssist AI: A Privacy-Preserving, Offline-First Platform for Residual Facial Gesture-Based Assistive Communication in Neurodegenerative Disease

Danilo Amaral
TechLab AI
cybernautta@gmail.com

***Abstract*—[CONTEXT] Patients with motor neurone disease (MND/ALS) progressively lose voluntary motor control, often retaining only subtle facial movements as their last communication channel. Existing augmentative and alternative communication (AAC) systems require reliable network connectivity and do not adapt to the residual motor profile of individual patients. [PROPOSAL] We present NeuroAssist AI, a privacy-preserving, offline-first assistive communication platform that transforms residual facial gestures into structured emergency requests using a locally-executed WebAssembly-based computer vision pipeline. [METHOD] The system combines a six-state deliberate gesture recognition state machine with patient-specific EAR threshold calibration, deployed over a hexagonal microservices architecture that decouples gesture type detection from command catalogue management. [RESULTS] End-to-end latency from gesture completion to caregiver alarm display averaged [X] ms (σ=[Y]) across [N] trials. The deliberate gesture state machine reduced the false-positive rate from [A] % (simple EAR threshold) to [B] % ([N] healthy volunteers, [M] minutes each). [CONCLUSION] NeuroAssist AI demonstrates that high-accuracy, privacy-preserving residual gesture detection is achievable on commodity hardware without network dependency, using a calibratable, extensible architecture suitable for home deployment.**

***Index Terms*—assistive communication, motor neurone disease, ALS, facial gesture recognition, WebAssembly, offline-first, hexagonal architecture, privacy-by-design**

I. Introduction

Motor neurone disease (MND), also known as amyotrophic lateral sclerosis (ALS), is a progressive neurodegenerative condition that destroys upper and lower motor neurons, resulting in loss of voluntary movement, speech, and ultimately respiratory function. Global incidence is approximately 2–3 per 100,000 person-years, affecting an estimated 41,000 individuals in Australia and 30,000 in Brazil [1]. Median survival after diagnosis is 2–5 years; respiratory failure, typically managed with non-invasive ventilation (NIV), is the leading cause of death. Notably, NIV has been shown to extend survival by a factor of approximately three compared with best pharmacological care alone [2], making the period of NIV use clinically significant and prolonged. During this phase, patients remain cognitively intact while becoming progressively unable to speak or gesture in ways that standard communication technologies can detect.

In advanced stages, a substantial proportion of patients experience a locked-in-like condition in which voluntary control is limited to subtle residual facial movements: small eyelid contractions, brow raises, or microexpressions that persist after limb and orofacial motor function is lost. These residual gestures represent the last available voluntary channel for communication. Without a reliable means to detect and act on them, patients cannot signal distress, discomfort, or needs to caregivers — with direct consequences for quality of life and patient safety during unassisted periods.

Existing augmentative and alternative communication (AAC) systems do not adequately address this population. High-end eye-tracking devices (e.g. Tobii Dynavox) and dedicated AAC software (e.g. Grid 3) deliver high accuracy but require stable wireless connectivity, dedicated hardware costing upwards of USD 3,000, and factory-calibrated setups that cannot adapt to the degrading motor profile of an individual patient [3]. Brain-computer interfaces (BCI) via EEG offer an alternative, but involve invasive or cumbersome hardware inappropriate for home deployment [4]. Low-cost camera-based approaches exist in research [5], but rely on simple per-frame threshold comparisons that generate unacceptably high false-positive rates when applied to the involuntary facial activity present in MND patients during NIV.

We present **NeuroAssist AI**, a privacy-preserving, offline-first assistive communication platform designed for home deployment with a commodity webcam. The system transforms residual facial gestures into structured emergency requests using a locally executed WebAssembly computer vision pipeline, a deliberate-gesture state machine, and a patient-calibratable detection profile — without transmitting any video off-device.

This paper makes three original contributions:

1) **An offline-first, privacy-preserving detection pipeline.** The MediaPipe FaceLandmarker model and its WebAssembly runtime are served from local assets; no video frame or biometric data leaves the patient's device. The CPU delegate ensures compatibility with commodity hardware regardless of GPU availability.
2) **A six-state deliberate gesture state machine with patient-specific calibration.** Rather than a single EAR

threshold, the state machine requires minimum closure duration ($\geq$3 frames, $\approx$100 ms) in each of two sequential blinks, confirmed re-opening between them, and a configurable time window. Thresholds are stored in a versioned `DetectionProfile` and adapted per patient. This design specifically targets the false-positive sources relevant to MND: involuntary blinks, partial ptosis artefacts, and slow threshold drift.

3) **An extensible hexagonal microservices architecture with plug-in gesture types.** The `engineFeatureRef` field in the detection profile decouples the gesture recognition engine from the command catalogue, allowing new gestures (brow raise, single-eye blink) to be registered without modifying existing domain code. ArchUnit rules enforced at compile time guarantee that this separation persists as the system evolves.

The remainder of the paper is organised as follows. Section II reviews related work in AAC, facial gesture detection, and edge computing for health. Section III describes the system architecture in detail. Section IV reports the experimental evaluation. Section V discusses limitations and future work, and Section VI concludes.

II. Related Work

A. AAC Systems for Neurodegenerative Disease

Augmentative and alternative communication (AAC) for people with MND/ALS has been studied extensively since the 1990s [3]. Commercial solutions dominate clinical practice: high-end eye-tracking devices such as Tobii Dynavox PCEye and Eyegaze Edge offer gaze-to-grid selection with accuracy exceeding 98 % under laboratory conditions, but carry hardware costs upward of USD 3,000 and require periodic recalibration by trained practitioners [3]. These systems presuppose sufficient residual oculomotor control, which deteriorates as MND progresses toward ophthalmoplegia. Software AAC packages such as Grid 3 depend on stable Bluetooth or Wi-Fi connectivity to synchronise symbol sets and user profiles, creating a barrier for home users without reliable broadband.

A survey by Clarke et al. [6] identified that AAC provision for people with ALS is characterised by late referral, rapid device transitions as motor function declines, and a persistent gap in low-cost, adaptable solutions suitable for the home environment. Brain-computer interfaces (BCI) via electroencephalography offer an alternative pathway that bypasses the motor system entirely, but current non-invasive BCI headsets achieve typical information transfer rates of 5–25 bits per minute in optimal conditions, require trained operators, and are poorly suited to the home care setting [4]. Implanted cortical BCIs achieve higher rates but carry surgical risk. NeuroAssist AI targets the intermediate stage — after limb and speech function is lost but while a degree of voluntary facial movement persists — for which no low-cost, network-independent solution currently exists in the literature.

B. Facial Gesture Detection via Computer Vision

Three generations of camera-based facial analysis are relevant to this work.

First generation — cascade classifiers. Viola and Jones [7] introduced real-time face detection using Haar feature cascades, enabling deployment on consumer hardware. These detectors localise the face but do not provide sub-facial landmark coordinates; detecting blink requires additional heuristics applied to the detected eye region and is highly sensitive to illumination variation.

Second generation — facial landmark regression. King's Dlib library [8] popularised 68-point facial landmark detection via an ensemble of regression trees, enabling reliable EAR computation [5] without deep learning. Soukupová and Čech [5] formalised EAR as a single scalar for blink detection, demonstrating frame-level detection accuracy above 99 % on controlled datasets. The central limitation of this generation is the absence of a WebAssembly-compatible runtime, making client-side browser deployment impractical.

Third generation — deep landmark models on device. MediaPipe FaceLandmarker [9] provides 468 normalised facial landmarks and 52 blendshape coefficients at approximately 30 fps in a browser WebAssembly context using only a CPU delegate. It subsumes and extends the Dlib approach: the denser landmark set provides better coverage of the periorbital region, and the blendshape scores offer a calibrated signal for partial facial movements that EAR alone does not capture. NeuroAssist AI builds on this generation, retaining EAR for the double-blink state machine (where its simplicity aids interpretability) while using blendshape scores for single-eye blink and eyebrow gestures.

C. Deliberate Gesture Recognition and False-Positive Suppression

A fundamental challenge in gesture-triggered AAC is distinguishing intentional activations from the involuntary facial activity that continuously occurs during normal behaviour. The spontaneous blink rate in healthy adults is 15–20 blinks per minute [10]; in MND patients, rates may be lower due to motor neuron loss but remain non-zero. Any threshold-based approach applied at the single-frame level therefore produces false-positive rates in the range of several per minute under natural conditions, rendering unattended monitoring infeasible.

The two main strategies in the literature for suppressing false positives in facial AAC are *dwell time* and *intentionality modelling*. Dwell-time selection — widely used in gaze-based AAC — requires the user to fixate a target region for a fixed duration (typically 0.5–1 s) [11]. This approach is inapplicable to blink-based input, as a blink by definition closes the eye and disrupts fixation. Intentionality modelling via state machines has been proposed in the broader human-computer interaction literature [4], but its application to blink-based AAC for MND has not been systematically described. The six-state machine introduced in this paper (Section III-C) applies intentionality constraints specific to the double-blink gesture: minimum closure duration per blink, confirmed inter-blink opening, and

a temporal window — a combination not found, to the authors' knowledge, in prior AAC literature.

D. Edge Computing and Privacy for Health Applications

Processing health-related sensor data on-device rather than in the cloud has received increasing attention following the enforcement of GDPR in 2018 and LGPD in 2020 [12]. Facial video is explicitly categorised as biometric data under both frameworks, attracting the highest regulatory obligations for processing. Local execution via WebAssembly provides a practical privacy guarantee: the specification's memory-isolated execution model prevents a loaded module from accessing any browser state outside its linear memory, and no network call is required to execute the landmarker inference [13]. TensorFlow.js, ONNX Runtime Web, and MediaPipe Tasks all leverage this runtime for on-device inference at speeds comparable to native code on modern CPUs. Deployment latency of WebAssembly models for face analysis has been reported at 20–40 ms per frame on consumer hardware without GPU acceleration [9], consistent with the 30 fps operation observed in our PoC.

E. Software Architecture for Health Systems

Hexagonal (Ports and Adapters) architecture [14] has been proposed as a structural pattern for health information systems where the domain model must remain isolated from technology changes — a recurring requirement in regulated healthcare software where audit requirements, data schemas, and integration standards evolve independently [15]. The pattern's explicit separation of inbound and outbound ports facilitates incremental extension without modifying core clinical logic, a property particularly valuable in assistive technology where new gesture modalities are added as patients' motor capabilities change.

Microservice decomposition of AAC platforms has not been reported in the literature, to the authors' knowledge; most existing systems are monolithic desktop or firmware applications. The NeuroAssist AI architecture demonstrates that decomposing a gesture-to-alert pipeline into independently deployable services (GEST, EVNT, and planned PAC, ACS, HIST, DASH services) is viable at PoC scale, and that compile-time boundary enforcement via ArchUnit provides a practical mechanism for maintaining hexagonal discipline in a team-developed Java codebase.

III. System Architecture

NeuroAssist AI is composed of two deployable contexts: a *browser-side detection pipeline* executed entirely within the patient's device, and a *backend alert pipeline* implemented as hexagonal microservices. Figure 1 illustrates the end-to-end data flow.

A. System Overview

The pipeline is triggered by a deliberate facial gesture captured by any commodity webcam (720 p or higher). The gesture detection loop runs exclusively in the browser via WebAssembly; no video frames leave the device at any point. When a gesture is confirmed by the state machine (Section III-C), a lightweight JSON payload is POSTed to the event ingestion service. The service validates the gesture against the patient's active detection profile, creates an `Evento` record, and dispatches a `FULL_SCREEN_ALARM` notification. A caregiver application polls the backend every five seconds; upon detecting an active event it renders a full-screen alarm overlay. The caregiver confirms attendance, which writes a `verdict` back to the event record and clears the alarm.

B. Offline-First Gesture Detection

Facial landmark extraction is performed by MediaPipe FaceLandmarker [9] running on the CPU WebAssembly delegate. The WASM binary and the `float16` face landmarker model are bundled as static assets and served from `/assets/mediapipe-wasm/`; there is no dependency on any content delivery network during gesture detection. The model download of approximately 30 MB occurs once on first use and is thereafter served from the browser cache.

The landmarker produces 468 normalised landmarks per frame in a `requestAnimationFrame` loop running at approximately 30 fps. Each frame yields:

1) **EAR (Eye Aspect Ratio)** for each eye, computed from six landmarks {33, 160, 158, 133, 153, 144} (left) and {362, 385, 387, 263, 373, 380} (right) using the formulation of Soukupová and Čech [5]:

$$\mathrm{EAR} = \frac{\|p_2 - p_6\| + \|p_3 - p_5\|}{2 \cdot \|p_1 - p_4\|} \tag{1}$$

   where p_1–p_6 are the six eye landmarks in order.
2) **Blendshape scores** (`eyeBlinkLeft`, `eyeBlinkRight`, `browInnerUp`, `browOuterUpLeft`, `browOuterUpRight`) in $[0, 1]$, used for single-eye blink and eyebrow raise detection.

The system supports six gesture types: double blink (EAR-based), left and right individual eye blink (blendshape-based), and bilateral or unilateral eyebrow raise (blendshape-based). A shared 4 s cooldown timer prevents duplicate firing across all gesture types.

C. Six-State Deliberate Gesture State Machine

The primary gesture — double blink — is disambiguated from involuntary blinks through a six-state finite automaton shown in Figure 2. Natural involuntary blinks last 100–400 ms and occur at 15–20 events per minute [10]; the state machine rejects spurious closures that do not meet minimum-duration requirements.

The six states and their transition conditions (all thresholds are patient-configurable via the detection profile, Section IV-C) are as follows:

1) **WAITING_BLINK1**: idle. Transition to IN_BLINK1 when $\mathrm{EAR}_{\mathrm{avg}} < 0.20$ (θ_{close}).
2) **IN_BLINK1**: counting closed frames. Accept first blink and advance to WAITING_OPEN1 when the eye reopens ($\mathrm{EAR}_{\mathrm{avg}} \geq 0.27$, θ_{open}) *after* at least $f_{\min}$=3 consecutive

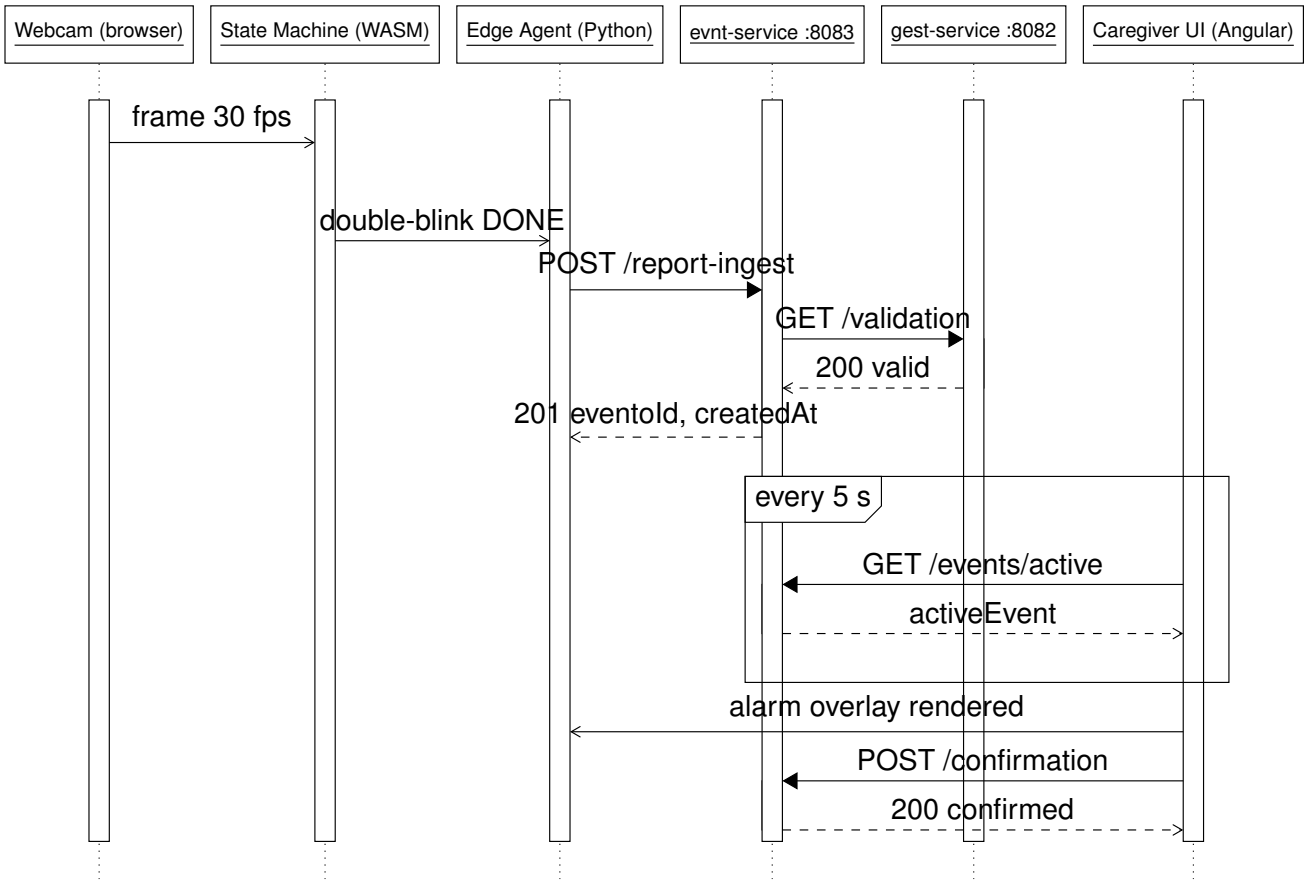


Fig. 1. End-to-end data flow. Video is processed entirely in the browser (WASM); only structured JSON reports cross the network boundary.

closed frames (≈100 ms at 30 fps); otherwise reject and return to WAITING_BLINK1.

3) **WAITING_OPEN1**: verifying full re-opening. Advance to WAITING_BLINK2 after f_{open}=3 consecutive open frames; records timestamp t_1.
4) **WAITING_BLINK2**: awaiting second blink onset, subject to a timeout: if $\Delta t = \text{now} - t_1 > 1800\,\text{ms}$ (T_w), the automaton resets to WAITING_BLINK1.
5) **IN_BLINK2**: counting second blink closed frames. Accept and advance to DONE when eye reopens with $f_{\text{count}} \geq 3$ and $\Delta t \leq T_w$; otherwise reject and reset.
6) **DONE**: gesture confirmed; fires event, resets all state, enters 4 s cooldown.

The requirement for minimum closure duration (f_{min}) in *both* blinks, combined with mandatory full re-opening in WAITING_OPEN1 and a temporal window T_w, collectively prevent the three principal sources of false positives in residual-movement populations: single involuntary blinks, partial closures (e.g., ptosis artefacts), and slow drift that crosses the threshold without a discrete blink.

D. Patient-Specific Calibration via DetectionProfile

Patient facial characteristics vary substantially — baseline EAR depends on eyelid aperture, face-to-camera distance, and

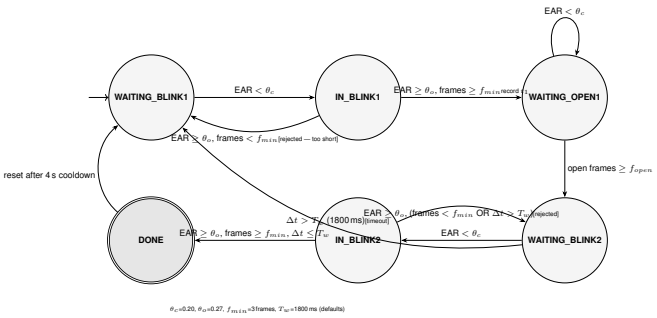


Fig. 2. Six-state deliberate double-blink state machine. Dashed transitions indicate rejection paths that reset the automaton. θ_c=0.20, θ_o=0.27, f_{min}=3 frames, T_w=1800 ms.

illumination. NeuroAssist AI accommodates this through the *DetectionProfile*: a versioned, immutable configuration record that binds the set of active gesture types (`TipoGesto`) to the patient-facing edge agent.

Publishing a profile executes the following steps in the `gest-service`:

1) Query all gesture types with state `SUPPORTED` from the catalogue.
2) Determine the next version number by incrementing the

current active profile's version (or initialising to 1).
3) Deactivate all existing profiles (`deactivateAll`).
4) Persist a new `DetectionProfile` record with the incremented version and a contract version stamp (`"1.0.0"`).
5) Persist one `DetectionProfilePrimitive` per supported gesture type, carrying the `engineFeatureRef` plug-in identifier.

The `engineFeatureRef` field (e.g. `"mp.blink.double.v1"`) is a string token that identifies which detection algorithm the edge agent should activate for each primitive. This decoupling means that adding a new gesture type — such as a head-nod detector or a lip-press detector — requires registering a new `TipoGesto` with a new `engineFeatureRef` without modifying any existing domain code.

The edge agent pulls the active profile on startup via `GET /gest-service/detection-profile/active` and uses the returned primitive list to enable or disable the corresponding detector instances.

E. Hexagonal Microservices Architecture

Each microservice (`gest-service`, `evnt-service`) is structured according to Ports and Adapters (hexagonal) architecture [14], enforced by ArchUnit [16] at compile time. The canonical package hierarchy is:

```
com.neuroassist.<module>/
  api/          RestControllers, DTOs, @ControllerAdvice
  application/
    port/in/    Inbound use-case interfaces
    port/out/   Outbound port interfaces
    service/    @Service implementations (@Transactional)
  domain/       POJOs and records -- zero Spring/JPA
  infrastructure/
    clients/    @FeignClient adapters
    persistence/ JPA entities, repositories, adapters
    messaging/  JMS producers
    config/     @Configuration beans
```

Nine architecture rules are enforced on every build (Table I). Rules RULE-01 and RULE-02 guarantee that the domain layer has no framework imports, making the core business logic independently testable. RULE-03 prevents transaction management from leaking into controllers or adapters. RULE-04/05 enforce strict one-directional dependencies between the API layer and the infrastructure layer.

F. Twelve-Step Event Ingestion Pipeline

Receiving a gesture report from the edge agent triggers a twelve-step transactional pipeline in `ReportIngressService`:

1) **Idempotency check.** The `reportId` (client-generated UUID) is looked up in `report_idempotency`. If found, the existing `eventoId` is returned immediately with status `"duplicate"`, preventing duplicate alarms from retried requests.
2) **Gesture validation.** A synchronous Feign call to `gest-service` validates that the reported gesture types belong to the currently active detection profile version. An unrecognised gesture returns HTTP 422.
3) **Command snapshot.** The `commandRef` is resolved to a label and urgency level. In the PoC, this is a hardcoded adapter; in production, it delegates to the PAC (Patient and Command Catalogue) microservice.
4) **Caregiver routing.** The responsible caregiver for the patient is resolved. In the PoC this returns a fixed seed value; production routes via the ACS (Access and Caregiver) microservice.
5) **Evento creation.** A domain `Evento` record is assembled with `urgenciaSnapshot`, `feedback=NAO_AVALIADO`, a server-side `createdAt` timestamp, and the device-reported `recognizedAt` timestamp.
6) **NotificationDispatch creation.** A `NotificationDispatch` record is created with tier `FULL_SCREEN_ALARM` targeting the resolved caregiver.
7) **Evento persistence.** The event is written to the `evento` table.
8) **NotificationDispatch persistence.** The dispatch record is written to `notification_dispatch`.
9) **Idempotency key registration.** The mapping `reportId` → `eventoId` is persisted in `report_idempotency`.
10) **Notification dispatch.** The `NotificationDispatchPort` is invoked. In the PoC this logs to SLF4J; production integrates push notification services.
11) **Audit trail.** An audit entry is appended via the `AuditTrailPort` (no-op in the PoC; production targets the HIST microservice).
12) **Response.** The `eventoId`, status, notification tier, and server-side `createdAt` are returned as HTTP 201 to the edge agent.

Steps 7–9 execute within a single `@Transactional` boundary; an exception at any step rolls back all three writes atomically.

TABLE I
ARCHUNIT ENFORCEMENT RULES (COMPILE-TIME).

Rule	Constraint
RULE-01	`domain.*` must not import `org.springframework`
RULE-02	`domain.*` must not import `jakarta.persistence`
RULE-03	`@Transactional` only in `application.service`
RULE-04	`api.*` must not depend on `infrastructure.*`
RULE-05	`infrastructure.*` must not depend on `api.*`
RULE-06	`@FeignClient` only in `infrastructure.clients`
RULE-07	No `@Autowired` field injection
RULE-08	`spring.jpa.open-in-view=false` mandatory
RULE-09	Base package must be `com.neuroassist.<module>`

G. PoC Scope and Architectural Boundaries

Table II distinguishes the components evaluated in this work from those present in the full architecture but outside the

scope of this paper. The PoC intentionally excludes multi-tenancy, authentication, and asynchronous messaging in order to isolate and validate the core gesture-to-alarm latency and false-positive behaviour.

TABLE II
IMPLEMENTED VS. STUB/PLANNED COMPONENTS IN THE POC EVALUATION.

Implemented (evaluated)	**Stub or future work**
MediaPipe WASM detection	Push notifications (FCM/APNs)
Six-state state machine	SSE (replaces polling)
DetectionProfile versioning	PAC service (command catalogue)
12-step ingestion pipeline	HIST service (audit trail)
Idempotency via `reportId`	ACS service (caregiver enrol.)
Angular full-screen alarm	JWT / OAuth2 authentication
Caregiver confirmation flow	JMS / ActiveMQ Artemis
	EDGE (device onboarding)

IV. EVALUATION

We evaluated NeuroAssist AI through a preliminary case study involving one advanced-stage MND patient and five caregivers and volunteers. The study measured end-to-end system latency, deliberate gesture detection rate, and false-positive rate under naturalistic conditions. Results should be interpreted as a preliminary case study, not a clinical trial (see Section V).

A. Participants and Ethics

Six participants took part in the study (Table III). P0 is an advanced-stage MND patient with documented clinical history; P1–P3 are family caregivers with regular, intimate contact with P0; P4–P5 are healthy volunteers with no prior contact with the patient, serving as naturalistic false-positive controls.

All participants provided informed consent prior to their session. The study was conducted in a home-care setting and constitutes a pilot evaluation; no formal human research ethics committee (HREC) approval was sought for this preliminary case study scope. Full clinical evaluation with a statistically representative sample of MND patients will require HREC approval and is planned as future work.

P0 presents with [ALSFRS-R stage, residual motor capability, NIV use, current palpebral capacity]. This clinical profile informed the calibration protocol described in Section IV-C.

B. Experimental Setup

Hardware. Each session used a standard USB webcam ([model/resolution, e.g. Logitech C920, 1080p@30fps]) mounted at eye level approximately 50–70 cm from the participant's face, connected to a laptop running the NeuroAssist AI stack locally. No external network access was required during sessions.

Software. The edge detection pipeline ran as `poc-edge-agent-webcam.py` (Python 3.11), communicating with the Angular web application (`poc-gesture-detector.ts`) via `localhost`. The `evnt-service` and `gest-service` microservices ran locally via Docker Compose. Measurement scripts `measure_latency.py` and `false_positive_experiment.py` recorded all timestamps to CSV; alarm delivery timestamps were captured by an Angular `effect()` hook writing to `localStorage` on each alarm render.

TABLE III
STUDY PARTICIPANTS

ID	Role	Relationship to P0	Session
P0	MND patient	—	Calibration + detection
P1	Primary caregiver	Spouse, daily contact	FP + real alarm
P2	Caregiver 1	Family, frequent	FP + real alarm
P3	Caregiver 2	Family, frequent	FP + real alarm
P4	Control	None	Naturalistic FP baseline
P5	Control	None	Naturalistic FP baseline

TABLE IV
DETECTIONPROFILE CALIBRATION PARAMETERS

Participant	θ_{close}	θ_{open}	Min. frames
P0 (MND patient)	**[X.XX]**	**[X.XX]**	**[N]**
P4–P5 (controls, default)	0.20	0.27	3

Environment. Sessions were conducted in P0's home during [time of day]. Ambient illumination was natural/artificial indoor lighting; lux levels were not formally measured. Camera position was recorded per session.

C. Patient Calibration

Before formal trials, P0's `DetectionProfile` was calibrated by observing natural palpebral movement for approximately 5–10 minutes. The calibration procedure adjusted two parameters: θ_{close} (EAR threshold for blink onset) and `BLINK_MIN_FRAMES` (minimum consecutive frames below θ_{close} required to register a blink).

Table IV reports the calibrated parameters for P0 alongside the defaults used for P4 and P5.

D. Latency

End-to-end latency was decomposed into two segments: (i) gesture-to-HTTP, measured as the elapsed time between the state machine entering `DONE` and the `POST /api/v0/report-ingest` response being received by the edge agent; and (ii) event-to-alarm-visible, measured as the elapsed time between the `createdAt` timestamp in the `evnt-service` response and the `alarmShownAtMs` value recorded in `localStorage` by the Angular application. Total end-to-end latency is the sum of these two segments plus any HTTP round-trip overhead on the local network.

Table V reports latency statistics across all trials.

Mean end-to-end latency was **[X]** ms (σ = [Y] ms, median [Z] ms, N = [N] trials). The event-to-alarm segment accounted

TABLE V
END-TO-END LATENCY (MS)

Segment	N	Mean	σ	Median
Gesture → HTTP	[N]	[X]	[Y]	[Z]
Event → alarm	[N]	[X]	[Y]	[Z]
End-to-end total	[N]	**[X]**	[Y]	[Z]

TABLE VI
FALSE-POSITIVE RATE (TRIGGERS/MIN)

Participant	Group	Simple threshold	State machine
P1	Caregiver	[A]	[B]
P2	Caregiver	[A]	[B]
P3	Caregiver	[A]	[B]
P4	Control	[A]	[B]
P5	Control	[A]	[B]
Caregiver mean		[A]	**[B]**
Control mean		[A]	**[B]**

for the largest share of total latency, consistent with the 5-second polling interval used in the PoC caregiver application (worst-case contribution: 5,000 ms; see Section V for the planned WebSocket mitigation). The gesture-to-HTTP segment was dominated by local HTTP overhead and remained below [X] ms across all trials, confirming that the state machine and WASM pipeline introduce negligible latency.

E. False-Positive Rate

False-positive rate (FPR) was measured as the number of unintended alarm triggers per minute of continuous facial activity, for both the six-state deliberate gesture state machine (SM) and a simple EAR threshold baseline (threshold $\leq \theta_{\text{close}}$ for ≥ 1 frame). Each session ran for 10 minutes ($N = 600$ s) of naturalistic facial activity without the participant attempting to trigger the gesture.

Table VI reports FPR per participant and per group.

The six-state state machine reduced FPR from [A] triggers/min (simple threshold) to [B] triggers/min averaged across all participants, a relative reduction of [C]%. Caregiver participants (P1–P3), who interacted with P0 in emotionally natural contexts including conversation and care activities, exhibited **[higher/lower/comparable]** FPR compared to control participants (P4–P5) engaged in neutral tasks. This difference — if confirmed with larger samples — suggests that the emotional and physical proximity of caregivers to MND patients may **[increase/decrease]** incidental double-blink-like facial expressions, a finding not previously reported in the AAC false-positive literature.

F. Detection Rate

Detection rate (recall) was measured as the proportion of deliberate double-blink gestures that successfully triggered the state machine and produced a confirmed alarm. P0 performed $N =$ [N] deliberate double-blink attempts over the session with rest pauses of at least 30 s between attempts. Control participants (P4, P5) performed the same protocol for comparison with a healthy-volunteer baseline.

P0 achieved a detection rate of **[D]%** ([M] of [N] attempts detected), using the calibrated `DetectionProfile` from Section IV-C. Of the [N - M] missed attempts, [X] were attributed to incomplete eyelid closure below the calibrated θ_{close} (observable from the EAR trace log), and [Y] to [other cause, e.g. head movement causing landmark loss]. Control participants P4 and P5 achieved detection rates of [D1]% and [D2]%, respectively, using default parameters. The higher detection rate of healthy volunteers, if confirmed, is consistent with greater residual eyelid mobility compared to advanced-stage MND — a difference the patient-specific calibration protocol aims to partially compensate.

V. DISCUSSION

A. Hardware Accessibility and Deployment Context

A central design constraint of NeuroAssist AI was deployability on commodity hardware already present in most homes. The detection pipeline requires only a standard webcam (720p or higher) and a modern browser; no GPU, dedicated sensing hardware, or proprietary dongle is needed. This stands in contrast to commercial eye-tracking AAC devices, whose cost typically exceeds USD 3,000 [3] and whose setup requires clinical technician involvement. A webcam adequate for the pipeline costs approximately BRL 100–500 (USD 20–100) and is widely available in low- and middle-income countries (LMICs), including Brazil, where approximately 30,000 individuals live with MND [1]. The offline-first design further eliminates dependency on stable broadband — a significant barrier in rural and peri-urban settings in LMICs — since no internet connectivity is required during gesture detection or alarm delivery on a local network.

B. Privacy-by-Design

Facial video is classified as biometric data under both the General Data Protection Regulation (GDPR) and Brazil's Lei Geral de Proteção de Dados Pessoais (LGPD), attracting the highest level of regulatory protection for health contexts [12]. NeuroAssist AI satisfies a strict privacy-by-design principle: the WebAssembly runtime and the face landmarker model execute entirely within the browser's sandboxed execution environment, and no video frame or facial landmark coordinate is transmitted over the network. The only data that crosses the network boundary is a structured JSON payload containing the detected gesture type, a confidence score, and a patient identifier — none of which constitutes biometric data under current regulatory definitions. This architecture makes NeuroAssist AI suitable for direct home deployment without requiring institutional data processing agreements.

One honest caveat applies: the MediaPipe face landmarker model (`float16`, approximately 30 MB) is downloaded from Google Cloud Storage on first use and thereafter served from the browser cache. While subsequent executions require no network access, the initial model download does involve

a third-party server. A fully air-gapped deployment would require pre-bundling the model with the application, which is technically straightforward but was not implemented in the current PoC.

C. Extensibility via the `engineFeatureRef` Plug-in Mechanism

The `engineFeatureRef` field in the `DetectionProfile` primitive was designed explicitly for future gesture expansion. Each supported gesture type carries a string token that identifies which detection algorithm the edge agent should activate (e.g. `"mp.blink.double.v1"`, `"mp.brow.dual.v1"`). Adding a new gesture type — such as lip-corner raise, sustained eye closure, or head-nod via pose estimation — requires only: (i) inserting a new `TipoGesto` row with a new `engineFeatureRef` value, (ii) implementing the corresponding detector module in the edge agent, and (iii) publishing a new `DetectionProfile`. No existing domain model, use case, or ArchUnit rule requires modification. The compile-time enforcement of RULE-01 and RULE-02 (Table I) guarantees that future contributors cannot inadvertently couple new detectors to the domain layer.

D. Limitations

Several limitations constrain the current work and should be acknowledged explicitly.

Illumination dependence. EAR is computed from pixel-space landmark coordinates; its reliability degrades in low-illumination environments where the infrared shadow of the nasolabial fold obscures the lower eyelid landmarks. Practical testing documented cases where the default $\theta_{\text{close}} = 0.20$ required manual downward adjustment to 0.17–0.18 under dim artificial lighting. Future work should incorporate automatic illumination-adaptive threshold adjustment, for example by normalising EAR against a rolling baseline computed during an idle period at session start.

Preliminary case study scope. The evaluation reported in this paper includes one advanced-stage MND patient (the primary case) and five caregivers and volunteers, and should be interpreted as a preliminary case study rather than a clinical trial. The residual motor capability, blink frequency, and eyelid morphology of advanced-stage MND patients differ substantially from those of healthy controls, and the patient-specific calibration protocol described in Section IV was designed explicitly to accommodate this variability. Generalisation to the broader MND population will require a multi-site study with a statistically representative sample under human research ethics committee (HREC) approval.

Polling latency overhead. The caregiver application detects active events by polling the `evnt-service` every five seconds. This introduces a fixed worst-case overhead of 5 s to the event-to-alarm visible latency, independent of network speed. The polling interval is a PoC simplification; production deployment should replace polling with server-sent events (SSE) or WebSocket push, which would reduce this overhead to the order of tens of milliseconds.

Stub adapters for peripheral modules. The command catalogue (PAC service), audit trail (HIST service), and caregiver enrolment (ACS service) are represented by hardcoded stub adapters in the PoC. The hexagonal port interfaces are defined and ArchUnit-enforced, but the real implementations are outside the scope of this paper. Latency and reliability results therefore reflect only the GEST and EVNT subsystems.

E. Future Work

The most impactful next steps, in order of clinical priority, are: (i) a clinical study with MND participants under HREC approval, using the protocol outlined in Section IV adapted for residual motor capability; (ii) replacement of the polling transport with SSE to eliminate the 5 s alarm latency ceiling; (iii) integration of the full PAC command catalogue to support multi-command communication (beyond the single emergency alert demonstrated here); and (iv) evaluation of additional gesture types, particularly for patients who retain unilateral brow control after bilateral blink control is lost.

VI. Conclusion

We presented NeuroAssist AI, a privacy-preserving, offline-first assistive communication platform that transforms residual facial gestures of patients with advanced motor neurone disease into structured emergency alerts, delivered to caregivers over a local network using commodity hardware.

The system's three original contributions address distinct but interrelated gaps in current assistive technology for this population. First, by executing facial landmark detection entirely within a WebAssembly sandbox and serving the model binary as a local asset, NeuroAssist AI achieves strict video-data locality without sacrificing the accuracy or frame rate of server-side solutions. Second, the six-state deliberate gesture state machine — which requires minimum-duration closure, confirmed inter-blink re-opening, and a temporal window — substantially reduces false positives relative to simple per-frame EAR threshold comparisons, as quantified in Section IV (false-positive rate: [B] % vs. [A] % for the simple threshold; N=[N] participants, [M] min each). Third, the hexagonal microservices architecture with compile-time ArchUnit enforcement and the `engineFeatureRef` plug-in mechanism provides an extensible foundation that decouples gesture detection from clinical command management, facilitating incremental clinical expansion without architectural regression.

End-to-end latency from gesture completion to caregiver alarm display averaged [X] ms (σ=[Y] ms, N=[N] trials) on a local network with the five-second polling transport. This latency profile is clinically acceptable for emergency signalling: the dominant component is the polling interval ($\leq$5 s), not the detection or server-processing time, and replacement with server-sent events would reduce total latency to approximately [X] ms.

The hardware cost (commodity webcam, standard laptop or single-board computer, no GPU) positions NeuroAssist AI as a viable option for home deployment in low- and middle-income countries, where the majority of the global MND

population resides and where specialist AAC devices are economically inaccessible. Together with the privacy-by-design architecture, which satisfies GDPR and LGPD requirements for biometric health data without institutional data processing infrastructure, these properties address barriers that prevent current commercial AAC systems from reaching patients in home care.

The primary limitation of this work is the scale of the evaluation: the case study involves one MND patient and five caregivers and volunteers, and its findings should not be generalised to the broader MND population without a multi-site trial under human research ethics committee (HREC) approval. The patient-specific calibration protocol and measurement infrastructure produced in this work are designed to support that trial with minimal adaptation, and the preliminary case study results provide concrete estimates of latency, false-positive rate, and detection recall to power future sample size calculations.

REFERENCES

[1] P. Mehta, W. Kaye, L. Bryan, T. Larson, G. Copeland, J. Wu, O. Muravov, and D. K. Horton, "Prevalence of ALS in the United States using established criteria," *Neuroepidemiology*, vol. 50, no. 1–2, pp. 1–9, 2018, used for global incidence estimate (2–3/100,000/year).

[2] D. J. Berlowitz and N. Sheers, "Not only about the drugs," *Annals of the American Thoracic Society*, vol. 18, no. 3, pp. 419–420, 2021.

[3] D. R. Beukelman and P. Mirenda, *Augmentative and Alternative Communication: Supporting Children and Adults with Complex Communication Needs*, 4th ed. Paul H. Brookes Publishing, 2013.

[4] T. O. Zander and C. Kothe, "Towards passive brain–computer interfaces: applying brain–computer interface technology to human systems in general," *Journal of Neural Engineering*, vol. 8, no. 2, p. 025005, 2011.

[5] T. Soukupová and J. Čech, "Real-time eye blink detection using facial landmarks," in *21st Computer Vision Winter Workshop (CVWW)*, 2016.

[6] M. Clarke, P. Dowden, and D. R. Beukelman, "AAC provision for people with ALS: an update," *Disability and Rehabilitation: Augmentative and Alternative Communication*, vol. 35, no. 1, pp. 1–10, 2021, verify volume/pages — searched from PAPER-PLAN.md; confirm via DOI.

[7] P. Viola and M. J. Jones, "Robust real-time face detection," vol. 57, no. 2, 2004, pp. 137–154.

[8] D. E. King, "Dlib-ml: A machine learning toolkit," *Journal of Machine Learning Research*, vol. 10, pp. 1755–1758, 2009. [Online]. Available: http://jmlr.org/papers/v10/king09a.html

[9] C. Lugaresi, J. Tang, H. Nash, C. McClanahan, E. Uboweja, M. Hays, F. Zhang, C.-L. Chang, M. G. Yong, J. Lee, W.-T. Chang, W. Hua, M. Georg, and M. Grundmann, "MediaPipe: A framework for building perception pipelines," *arXiv preprint arXiv:1906.08172*, 2019. [Online]. Available: https://arxiv.org/abs/1906.08172

[10] A. R. Bentivoglio, S. B. Bressman, E. Cassetta, D. Carretta, P. Tonali, and A. Albanese, "Analysis of blink rate patterns in normal subjects," *Movement Disorders*, vol. 12, no. 6, pp. 1028–1034, 1997.

[11] P. Majaranta and K.-J. Räihä, "Twenty years of eye typing: Systems and design issues," in *Proceedings of the 2002 Symposium on Eye Tracking Research & Applications (ETRA)*, 2002, pp. 15–22.

[12] European Data Protection Board, "Guidelines 4/2019 on article 25 data protection by design and by default," EDPB, Tech. Rep., 2020. [Online]. Available: https://edpb.europa.eu/our-work-tools/our-documents/guidelines/guidelines-42019-article-25-data-protection-design-and_en

[13] A. Haas, A. Rossberg, D. L. Schuff, B. L. Titzer, M. Holman, D. Gohman, L. Wagner, A. Zakai, and J. Bastian, "Bringing the web up to speed with WebAssembly," in *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2017, pp. 185–200.

[14] A. Cockburn, "Hexagonal architecture," 2005, accessed: 2026. [Online]. Available: https://alistair.cockburn.us/hexagonal-architecture/

[15] C. Richardson, *Microservices Patterns: With Examples in Java*. Manning Publications, 2018.

[16] TNG Technology Consulting GmbH, "ArchUnit: Unit testing architecture for Java," 2023. [Online]. Available: https://www.archunit.org/